BrowserBox Customer Guide

DOSAYGO

Current as of BrowserBox v18.3.1
July 28, 2026

Status: Provisional customer reference. Supported commands, environment variables, login-link parameters, and operational patterns are subject to change in future BrowserBox releases.

Contents

1 Purpose

This document combines the customer-facing BrowserBox setup guide and the customer-relevant configuration reference into one printable manual. It is intended to answer the questions customers usually ask first:

- How do I install BrowserBox on macOS, Linux, or Windows?
- How do I set BrowserBox up and start it?
- What can I customize?
- How do reverse proxy, HTTP mode, and certificates work?
- How do I embed BrowserBox safely in my own application?
- What do the tunnel-oriented commands mean?
- What licensing visibility exists today?

2 Installation and Customer Resources

2.1 Official install entrypoints

BrowserBox installation guidance is published at https://browserbox.io. The current public install entrypoints are:

- macOS and Linux: `curl -fsSL https://browserbox.io/install.sh | bash`
- Windows: `irm https://browserbox.io/install.ps1 | iex`

The Windows installer exists and is useful for getting a BrowserBox binary onto a Windows machine, but the Windows PowerShell `bbx` surface is not command-parity compatible with the bash-oriented `bbx` tool documented in this guide. For that reason, this guide documents the bash CLI behavior and uses the Windows path only as an installation entrypoint.

2.2 Suggested first run

After installation, the normal customer workflow is:

```
bbx certify YOUR_KEY
bbx setup -p 8888
bbx start
```

If you are evaluating BrowserBox, obtain your license through https://browserbox.io. Commercial licensing and evaluation paths are surfaced there, and the site links through to the current purchase flow.

2.3 Prerequisites

BrowserBox requires a Chrome-family browser (Google Chrome or Chromium) installed on the host machine. If Chrome is not found, runtime commands (`start`, `run`, `ng-run`, etc.) will refuse to launch and print:

```
Chrome/Chromium is not installed.
BrowserBox requires a Chrome-family browser to run.
Install it with:  browserbox --full-install <hostname> <email>
```

2.4 Selecting or pinning Chrome

BrowserBox normally discovers a compatible Chrome-family browser automatically. When support recommends a specific Chrome major for compatibility testing, use:

```
bbx use-chrome 149
```

The command resolves and installs the requested Chrome-for-Testing major, then persists the resulting `CHROME_PATH` in `~/.config/dosaygo/bbpro/user.env`. The selection therefore survives `bbx setup` and service restarts. You may also supply an executable path directly, for example `bbx use-chrome /opt/google/chrome/chrome`.

Pinning is a compatibility or rollback measure, not the first response to every startup failure. Before changing Chrome, run `bbx logs` and check for integrity, configuration, license, port, or certificate errors. To return to automatic browser discovery, remove the `CHROME_PATH` line from `user.env` and run `bbx restart`.

2.5 Full machine setup (`-full-install`)

On a fresh server without Chrome or other dependencies:

```
browserbox --full-install <hostname> <email>
```

- `<hostname>` — the domain or IP address (e.g. `localhost`, `bbx.example.com`, `5.1.2.3`).
- `<email>` — email used for BrowserBox terms acceptance and, when applicable, public-certificate issuance.

This installs Chrome, Node.js, TLS tooling, and the other OS-level dependencies BrowserBox needs. For non-interactive installs, the preferred environment variables are `BBX_INSTALL_HOSTNAME` and `BBX_INSTALL_EMAIL`. Legacy aliases `BBX_HOSTNAME`, `BBX_EMAIL`, and `EMAIL` are still accepted. If you run the standalone installer as `root` in a non-interactive environment, also set `BBX_INSTALL_USER` to the non-root account that should own the installation.

For non-interactive (scripted) installs, set `BBX_TEST_AGREEMENT=true` to auto-agree to the license agreement:

```
export BBX_TEST_AGREEMENT=true
export BBX_INSTALL_HOSTNAME="bbx.example.com"
export BBX_INSTALL_EMAIL="you@example.com"
browserbox --full-install "$BBX_INSTALL_HOSTNAME" "$BBX_INSTALL_EMAIL"
```

2.6 Support and public references

- Product site and license purchasing: https://browserbox.io
- Public GitHub repository: https://github.com/BrowserBox/BrowserBox
- Support email: support@dosaygo.com

3 Fast Start

3.1 Standard local workflow

```
bbx stop
bbx setup -p 8888
bbx start
```

When you run `bbx setup -p 8888`, port `8888` becomes the main BrowserBox service port. Related service ports are derived from the main port:

- audio: main port - 2
- docs: main port - 1
- devtools: main port + 1

BrowserBox writes the current login link to:

```
~/.config/dosaygo/bbpro/login.link
```

3.2 Core commands

Command	Purpose
`bbx setup`	Configure port, hostname, token, zeta mode, and backend mode.
`bbx start`	Start BrowserBox for the current user.
`bbx stop`	Stop BrowserBox for the current user.
`bbx status`	Check whether BrowserBox appears reachable and running.
`bbx logs`	View BrowserBox logs.
`bbx certify`	Validate the current license and reserve a seat when applicable.
`bbx vacancy`	Show total, occupied, vacant, leased, and reserved seat counts, plus any local reservation metadata on the machine.
`bbx ng-run`	Run the nginx-oriented workflow.
`bbx tor-run`	Run BrowserBox with Tor integration.
`bbx zt-run`	Run BrowserBox on a ZeroTier network.
`bbx cf-run`	Run BrowserBox through a Cloudflare tunnel.

`bbx logs` displays the BrowserBox service list. To tail the main service directly, run:

```
browserbox pm2 logs bb-main --lines 50
```

The `browserbox pm2` command owns process-manager operations. Do not pass `pm2` to `bbpro`; `bbpro` expects its first argument to be an environment-file path.

4 Customer Customization Surfaces

4.1 Login-link URL parameters

BrowserBox generates login links with query parameters that affect customer-visible behavior.

Parameter	Meaning
`ui=false`	Hides the standard BrowserBox chrome for embedding or kiosk-style use.

`url=`*`json-url-payload`*	Opens target pages on first load. The value is parsed as JSON: use a JSON string for one starting URL, or a JSON array of strings for multiple starting URLs. URL-encode that JSON payload when placing it on the login link.
`mid=`*`machine-id`*	Infrastructure routing affinity in multi-machine deployments. This is not a user identity or a seat identifier.
`uiTheme=`*`theme`*	Requests a UI theme when the server accepts that theme value.

The `url=` payload is JSON-shaped, not a bare URL. For one starting page, use a JSON string such as "https://example.com". To open multiple starting tabs, use a JSON array such as ["https://example.com","https://docs.example.com"]. In the actual login-link query string, URL-encode that JSON value before appending it as `url=...`.

When reopening a login link for a running session, the `url=` parameter will intelligently reuse and navigate an existing suitable tab rather than creating a duplicate tab. This makes it fully compatible with single-tab topologies such as `BBX_MAX_TABS=1`.

When `ui=` is explicitly present, BrowserBox treats that as embedder-owned configuration and locks the user toggle accordingly.

4.2 Homepage selection

Use either of these variables:

```
export BBX_HOME_PAGE="https://intranet.example.com"
# or
export BBX_DEFAULT_HOME_PAGE="https://intranet.example.com"
```

By default, BrowserBox asks Chrome to restore tabs from the previous browser session. If no usable page tab is restored, BrowserBox opens the configured home page. To preserve the browser profile while always beginning with the home page instead of restored tabs, set:

```
export BBX_RESTORE_LAST_SESSION=false
export BBX_HOME_PAGE="https://intranet.example.com"
```

4.3 Declared startup tabs

Use `BBX_STARTUP_TABS` to declare the exact tab topology for the initial BrowserBox start. The value is a non-empty JSON array of absolute URLs:

```
export BBX_STARTUP_TABS='["https://intranet.example.com"]'
```

This starts BrowserBox with exactly one page tab without restoring tabs from the previous Chrome session. It preserves cookies, local storage, preferences, and other browser-profile data. It does not restrict later navigation or prevent the user from opening more tabs.

Multiple initial tabs are also supported:

```
export BBX_STARTUP_TABS='["https://intranet.example.com", "https://docs.example.com"]'
```

The number of declared startup tabs must not exceed `BBX_MAX_TABS`. Navigation policy still applies to every declared URL. When set, `BBX_STARTUP_TABS` takes precedence over `BBX_HOME_PAGE` and `BBX_RESTORE_LAST_SESSION` for initial startup.

The value must be a valid `http:` or `https:` URL. Invalid values fall back to https://duckduckgo.com.

4.4 Cross-origin iframe compatibility (OOPIF)

BrowserBox enables out-of-process iframe (OOPIF) support by default. Modern Chrome may place a cross-site iframe in a separate renderer process; BrowserBox attaches to that target so cursor support, page integrations, and supported injected capabilities continue to work inside the frame. This also applies recursively to nested cross-site frames.

If a site exhibits a compatibility or stability problem after an upgrade, an administrator can temporarily return to the previous behavior:

```
export BBX_OOPIF=false
bbx restart
```

For a persistent setting, add `BBX_OOPIF=false` to `~/.config/dosaygo/bbpro/user.env`, then restart BrowserBox. Disabling OOPIF support means BrowserBox will no longer attach to separately rendered cross-site iframe targets, so BrowserBox-provided cursor and page integrations may not be available inside those frames. Include `bbx logs` output when reporting an OOPIF compatibility issue.

4.5 Printing

BrowserBox handles printing as an explicit download intent rather than opening a printer attached to the BrowserBox host. When a page calls `window.print()`, the built-in Chrome PDF viewer requests printing, or the user selects **Print** from the BrowserBox context menu, BrowserBox asks for confirmation. After confirmation, BrowserBox creates or retrieves a PDF and presents the existing download-ready dialog. The user downloads that PDF and prints it with a local printer.

Printing requires downloads to be allowed by the active BrowserBox policy. A print request from a separately rendered OOPIF is captured in that target; a request from a same-process iframe produces the printable document for its owning page target.

4.6 Custom cursor

```
export BBX_CUSTOM_CURSOR="/absolute/path/to/cursor.png"
bbx start
```

BrowserBox reads the local file at startup and uses it as the remote cursor image.

4.7 UI theme override

```
export BBX_UI_THEME_OVERRIDE=dark
bbx start
```

This sets the default BrowserBox UI theme for the login flow. A login-link `uiTheme=` parameter can also request a theme when accepted by the server.

4.8 Idle shutdown after disconnect

```
export BBX_SHUTDOWN_IDLE_MS=7200000
bbx start
```

This controls how long BrowserBox waits after all clients disconnect before shutting down.

If you run with `BBX_MINIMAL_MODE="true"` and do not set `BBX_SHUTDOWN_IDLE_MS`, BrowserBox uses `BBX_MINIMAL_MODE_IDLE_MS` as the fallback idle shutdown duration.

5 Customer Policy Surface

5.1 What policy controls

BrowserBox policy is the customer-facing runtime control surface that answers two distinct questions:

- Which sensitive actions are allowed or denied?
- Which browsing targets may the remote browser reach?

The canonical persisted policy file lives at:

```
~/.config/dosaygo/bbpro/policy/policy.json
```

BrowserBox normalizes and validates that file before applying it. If a field is omitted, BrowserBox fills in safe defaults for the current schema.

The allowed values listed in this section are based on BrowserBox's current runtime validation and normalization surface, not on a separate aspirational specification. In practice, customers should be able to trust this section because these values are the same ones BrowserBox validates at load time.

5.2 Canonical policy fields

The customer-relevant policy bundle currently includes these high-level surfaces:

Field	Meaning
`transport.certificateErrors`	Whether BrowserBox enforces certificate validation or explicitly allows self-signed / invalid certificates.
`navigation.mode`	Top-level navigation mode: `open`, `allowlist`, or `blocklist`.
`navigation.allowlist`	Allowed navigation targets when `navigation.mode=allowlist`. Supports exact host, exact origin, exact URL, and wildcard host patterns.
`navigation.blocklist`	Denied navigation targets when `navigation.mode=blocklist`.
`navigation.privateNetwork.mode`	Whether private/local targets are denied by default or controlled through an allowlist.
`navigation.privateNetwork.allowlist`	Explicit private/local destinations that may be reached, including exact hosts, IPs, and IPv4 CIDRs.
`featureControls.copy`	Controls copy actions, including modifier shortcuts and context-menu copy paths.
`featureControls.paste`	Controls paste actions, including text insertion into the remote browser.
`featureControls.downloads`	Controls download routes and archive-serving surfaces. When set to `deny`, the download is cancelled at the browser level before any data is written to disk, and the connected client receives a policy-denied notification.
`featureControls.uploads`	Controls upload routes and file input surfaces.
`featureControls.devtools`	Controls DevTools access.
`featureControls.audio`	Controls the audio sidecar service.

`featureControls.automation`	Controls the BrowserBox automation surface used by the webview API.
`featureControls.contextMenu`	Controls whether the remote-page context menu may open.
`featureControls.browserChrome`	Controls BrowserBox's own visible browser chrome mode rather than allowing or denying an action.

5.3 Allowed values and defaults

The most important values customers need to know are listed here directly.

5.3.1 Binary feature controls

Unless stated otherwise, feature controls are binary decisions:

Field	Allowed values	Default / note
`featureControls.copy`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.paste`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.downloads`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.uploads`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.devtools`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.audio`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.automation`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.contextMenu`	`allow`, `deny`	If omitted, BrowserBox currently defaults this field to `allow` for backward compatibility.

5.3.2 Enumerated policy fields

Field	Allowed values	Default / note
`transport.certificateErrors`	`enforce`, `ignore-all`	Defaults to `enforce`.
`navigation.mode`	`open`, `allowlist`, `blocklist`	Required in the persisted policy bundle.
`navigation.privateNetwork.mode`	`deny`, `allowlist`	Defaults to `deny`.
`featureControls.browserChrome`	`toggle`, `api`, `always-show`, `always-hide`	Defaults to `toggle`.

5.3.3 Identity and access configuration

Three fields govern identity integration for authenticated BrowserBox deployments:

Field	Allowed values	Note
`identity.authMode`	`magic-link`, `sso-saml`	How users authenticate. `magic-link` uses email-based one-time links; `sso-saml` routes through an IdP.
`identity.ssoProvider`	`none`, `okta`, `entra`, `custom`	The IdP in use when `authMode=sso-saml`. Use `custom` for non-enumerated providers.
`identity.peerCapabilityModel`	`shared`, `scoped`	Whether multiple connected clients in the same session share capabilities (`shared`) or each client has its own scoped capability set (`scoped`).

5.3.4 List-valued policy fields

Field	Allowed entry form
`navigation.allowlist`	Array of non-empty strings. Entries may be exact hosts, exact origins, exact URLs, or wildcard host patterns.
`navigation.blocklist`	Array of non-empty strings. Entries follow the same matching forms as the navigation allowlist.
`navigation.privateNetwork.allowlist`	Array of non-empty strings. Entries may be exact hosts, exact IPs, or IPv4 CIDRs.

5.3.5 Where customers can verify allowed values

Customers do not have to guess. BrowserBox exposes the current policy surface through the customer-facing policy CLI:

- `bbx policy get` prints the persisted policy bundle.
- `bbx policy resolve` prints the normalized effective policy that BrowserBox is actually using.
- `bbx policy validate -file policy.json` validates a candidate policy file against the current code and reports invalid enum values directly.

For example, if a customer uses an unsupported mode, BrowserBox validation reports the current accepted set, such as:

```
featureControls.browserChrome must be one of:
toggle, api, always-show, always-hide
```

5.4 Sensitive actions BrowserBox can allow or deny

These actions currently route through BrowserBox's policy authorization layer:

Action	Customer-visible effect
navigate	Page navigation, history navigation, new-tab target creation, and clean-slate navigation flows.
uploads	File upload routes and file chooser / file input flows.
downloads	Download and archive-serving routes. When denied, the download is cancelled before any data reaches the server's download directory, and the user sees a policy-denied notification in the BrowserBox client UI.
devtools	BrowserBox DevTools service and its websocket bridge.
audio	BrowserBox audio service.
copy	Keyboard and context-menu copy paths.
paste	Keyboard and API-driven paste / insert-text paths.
automation	The automation methods exposed through the webview API, such as selector waits, clicks, typing, and evaluation.
contextmenu	The remote-page context menu UI.

All of these controls are binary `allow` / `deny` decisions except `featureControls.browserChrome`, which uses explicit UI visibility modes described below.

5.5 Navigation and private-network policy

BrowserBox separates public browsing control from private/local reachability:

- `navigation.allowlist` answers "where may the RBI tab browse?"
- `navigation.privateNetwork.allowlist` answers "which private or local destinations may any request reach?"
- `ALLOWED_EMBEDDING_ORIGINS` is separate and only controls which parent pages may embed BrowserBox

5.5.1 Navigation modes

Mode	Behavior
open	Any browsing target is allowed, subject to the separate private-network boundary.
allowlist	Only configured allowlist entries may be reached.
blocklist	All targets are allowed except those matching the configured blocklist.

Allowlist and blocklist entries support:

- exact host names, for example `example.com`
- exact origins, for example `https://secure.example.com`
- exact URLs, for example `https://secure.example.com/tools?mode=estimate`
- wildcard host patterns, for example `*.customer.example`

Note on path-prefix wildcards: patterns such as `google.com/maps/*` are *not* yet supported. Matching is currently against the host, origin, or full URL, not against a partial URL path. To permit a specific subdomain, use a subdomain entry such as `maps.google.com` instead of a path-prefix form. Path-prefix wildcard support is planned for a future release.

5.5.2 Private-network controls

Private-network policy defaults to deny. Customer policy may widen that boundary explicitly:

Field	Meaning
mode=deny	Denies private and local targets such as RFC1918 addresses and localhost by default.
mode=allowlist	Keeps the private/local boundary in place but permits explicit entries.
allowlist entries	Exact hosts, exact IPs, and IPv4 CIDRs, for example `10.0.0.5`, `10.0.0.0/8`, and `localhost`.

5.6 Certificate-error policy

Certificate handling is now policy-driven rather than hidden behind a permanent launch flag.

Mode	Behavior
enforce	Default. BrowserBox enforces certificate validation and will not silently accept certificate errors.
ignore-all	Explicit opt-in for environments that need to reach self-signed or otherwise invalid HTTPS targets.

5.7 Browser chrome policy modes

`featureControls.browserChrome` does not allow or deny a risky action. Instead, it governs whether BrowserBox's own visible browser chrome is shown, hidden, or user-toggleable.

Mode	Visible UI	User toggle	Meaning
toggle	visible by default	yes	Legacy-compatible mode. The user may show or hide BrowserBox chrome unless the embedder locks it.
api	visible by default	no	The embedder may drive UI visibility through the API, but the end user may not toggle it directly.
always-show	always visible	no	BrowserBox chrome must remain visible.
always-hide	always hidden	no	BrowserBox chrome stays hidden for kiosk-style or tightly embedded experiences.

If the embedder sets `ui-visible` or `allow-user-toggle-ui`, those values may further restrict the experience. They cannot widen what the server policy allows.

Navigation in always-hide mode: when chrome is hidden, BrowserBox's built-in back/forward buttons are not visible to the user. However, tab history is preserved internally. Embedders can navigate the remote browser's history programmatically through the webview API:

```
await webview.callApi('goBack');
await webview.callApi('goForward');
```

A context-menu back option is not currently available; it is planned for a future release. In the meantime, embedders that need end-user back navigation in always-hide mode should add their own back button wired to the `goBack` API call above.

5.8 Per-destination feature controls

`featureControls` settings such as `downloads` and `paste` are currently **global** — the same setting applies to all browsing destinations regardless of which site the user is on. Per-destination feature controls (different `featureControls` per allowlist entry) are planned but not yet supported. If your use case requires different download or clipboard behavior on different sites, the recommended approach is to run separate BrowserBox instances each with a distinct policy configuration.

5.9 Server policy versus embedder policy

BrowserBox has two relevant policy layers:

1. **Server policy**, stored in `policy.json`, which is authoritative.
2. **Embedder policy hints**, supplied through the canonical webview surface, which may only restrict and never widen the effective policy.

- Server policy is the floor.
- Embedder restrictions may narrow capabilities further.
- An embedder cannot override a server-side deny into an allow.

5.10 Customer policy CLI

BrowserBox ships a policy CLI so customers can inspect, initialize, validate, and test the effective policy surface.

```
bbx policy help
bbx policy where
bbx policy baselines
bbx policy controls [--json]
bbx policy get
bbx policy resolve
bbx policy check --action <name> [--url <https://...>] [--source <tag>]
bbx policy trace [--last <n>]
bbx policy init [--baseline regulated|compat]
bbx policy reset [--baseline regulated|compat]
bbx policy set --file <path/to/policy.json>
bbx policy validate [--file <path/to/policy.json>]
```

5.10.1 Useful commands

Command	Meaning
bbx policy where	Prints the canonical persisted policy path.
bbx policy baselines	Lists supported baseline names.
bbx policy controls –json	Prints the documented NIST 800-53 and FIPS 140 control surfaces.
bbx policy get	Prints the currently persisted customer policy bundle.
bbx policy resolve	Prints the compiled / normalized effective policy snapshot that BrowserBox is using.
bbx policy check –action ...	Evaluates a single action decision. Exit code `0` means allow; exit code `2` means deny.
bbx policy trace –last 20	Shows recent policy decisions from the decision trace log.
bbx policy init –baseline compat	Creates the initial policy file from a named baseline.
bbx policy reset –baseline regulated	Replaces the policy file with a named baseline.
bbx policy set –file policy.json	Loads and validates a customer-supplied policy file, then persists it.
bbx policy validate	Validates the current persisted policy or an explicitly supplied file.

5.11 Operational visibility

When policy blocks an action, BrowserBox emits both customer-visible and operator-visible signals:

- a **policy-denied notification in the client UI** when a connected client is present at the time of the block
- server log entries prefixed with `[policy-decision]`
- a persistent trace file at `~/.config/dosaygo/bbpro/policy/decisions.ndjson`

Actions that currently produce client-visible notifications include:

Action	Notification trigger
navigate	A navigation to a blocked destination.
downloads	A browser download that was cancelled before any bytes reached disk.
copy, paste, devtools, uploads, automation, contextmenu	The corresponding action was attempted while denied by policy.
tab-limit	A new tab or popup was blocked because the session is at the configured tab cap (`BBX_MAX_TABS`).

Tab-limit blocks that occur during service startup are silent—they close excess tabs without showing a client notification, since no client is connected yet.

6 Embedding and Origin Allowlisting

For the hosted BrowserBox SaaS service, the current REST API and embed reference is published at https://hyper-frame.art/api/. That reference covers both session-lifecycle REST endpoints and the canonical `<hyper-frame>` custom element using the current method and event names.

6.1 Server-side embedding allowlist

For self-hosted BrowserBox embeds, the main server-side allowlist is:

```
export ALLOWED_EMBEDDING_ORIGINS="https://app.example.com https://*.customer.example:*"
```

This controls which parent pages may embed BrowserBox.
Supported patterns include:

- exact origins, for example `https://app.example.com`
- subdomain wildcards, for example `https://*.customer.example`
- trailing port wildcards, for example `https://app.example.com:*`
- both together, for example `https://*.customer.example:*`

Important boundary:

- `ALLOWED_EMBEDDING_ORIGINS` controls which parent pages may embed BrowserBox.
- It does not control where the remote browser may browse.
- Browsing targets are controlled by BrowserBox policy.

6.2 Canonical embedder controls

For new embeds, prefer explicit webview controls over encoding everything into the login URL:

- `embedder-origin="https://app.example.com"`
- `ui-visible="false"`
- `allow-user-toggle-ui="false"`
- `first-load-cleanse="https://example.com"` when the embedder should remove inherited tabs and open a known first page

Minimal example:

```
<hyper-frame
  login-link="https://localhost:8888/login?token=..."
  embedder-origin="https://app.example.com"
  ui-visible="false"
  allow-user-toggle-ui="false"
></hyper-frame>
```

6.3 First-load tab cleansing

Some hosted or demo embedders need the first visible session state to be deterministic even when the remote browser already has tabs. The `first-load-cleanse` attribute runs once per `login-link`: it closes the current tab set when the webview API becomes ready, then optionally opens one replacement tab.

```
<hyper-frame
  login-link="https://localhost:8888/login?token=..."
  first-load-cleanse="https://duckduckgo.com"
></hyper-frame>
```

The value is intentional:

- a non-empty URL closes existing tabs, then opens that URL
- an empty value, such as `first-load-cleanse=""`, closes existing tabs and opens no replacement tab
- omitting the attribute leaves the existing session tabs alone

The public `firstLoadCleanse(url)` method has the same semantics for explicit embedder control. Passing a non-empty `url` opens that URL after the cleanse; passing an empty string performs the delete-only cleanse.

6.4 Embedder media permissions

Embedders that do not want BrowserBox to request microphone, camera, or display-capture access can set:

```
<hyper-frame
  login-link="https://localhost:8888/login?token=..."
  embedder-origin="https://app.example.com"
  media-permissions="none"
></hyper-frame>
```

The current values are:

Value	Meaning
`default`	BrowserBox uses its normal media behavior. This is also the fallback for missing, empty, or unknown values.
`none`	The embedder does not want BrowserBox to use microphone, camera, or display capture for this embedded session. The webview removes those features from the iframe `allow` attribute and syncs the setting into the BrowserBox client. Safari-family clients use this setting to skip the BrowserBox media explainer and the related preflight media request.

This is a coarse embedder media control. It is not a substitute for BrowserBox browsing policy, and it does not currently expose separate per-device settings such as audio-only or camera-only controls.

6.5 Embedder session unload warning

BrowserBox shows a `beforeunload` warning ("you are about to leave your remote browser") on the BrowserBox wrapper page itself when the end-user has interacted with the session. For embedders that own the outer page and do not want BrowserBox to interrupt the user's navigation of their own application, this can be suppressed:

```
<hyper-frame
  login-link="https://localhost:8888/login?token=..."
  embedder-origin="https://app.example.com"
  session-unload-warning="none"
></hyper-frame>
```

Value	Meaning
`default`	BrowserBox installs its wrapper-page `beforeunload` warning. This is also the fallback for missing, empty, or unknown values.
`none`	BrowserBox's wrapper-page `beforeunload` handler still runs, but is lazily evaluated at fire time: when the embedder's `session-unload-warning` is `none`, the handler refuses to block the unload. This design is race-free; the handler checks the current setting each time the user tries to navigate, so the embedder value does not need to win a setup-time race against the BrowserBox script load.

This setting does not affect the BrowserBox modal for `beforeunload` prompts originating from the remote page being browsed. Those are governed by the `beforeunload-behavior` attribute below.

6.6 Remote page beforeunload behavior

When a remote page being browsed inside BrowserBox attempts to block navigation via a `beforeunload` dialog, BrowserBox normally intercepts this and shows a modal to the user. For automated or programmatic use cases where navigation should never be blocked, this can be configured to automatically allow or block the navigation without showing a UI:

```
<hyper-frame
  login-link="..."
  beforeunload-behavior="leave"
></hyper-frame>
```

Value	Meaning
`default`	BrowserBox shows its standard modal, allowing the user (or the embedder via the Modal API) to decide.
`leave`	BrowserBox automatically clicks "Leave" (proceed with navigation) and does not show the modal UI.
`remain`	BrowserBox automatically clicks "Remain" (stay on page) and does not show the modal UI.

This design uses the same JiT-evaluation pattern as the session unload warning, making it robust against initialization races.

6.7 Modal lifecycle and dismissal API

BrowserBox modals are exposed to embedders using the same event convention as the rest of the webview surface: the BrowserBox frame sends a dashed message type with a data payload, and `<hyper-frame>` dispatches a `CustomEvent` whose `detail` is that payload. Where dotted aliases exist, they carry the same `detail` object.

The modal API should not require customers to learn separate methods for alert, confirm, prompt, file chooser, auth, or notice modals. Each modal is represented as data with an advertised list of actions. Customers either choose one of those actions or call the convenience dismissal method.

```
webview.addEventListener('modal-opened', event => {
  const modal = event.detail;
});

webview.addEventListener('modal-updated', event => {
  const modal = event.detail;
});

webview.addEventListener('modal-closed', event => {
  const result = event.detail;
});
```

For compatibility with the webview's existing alias style, the implementation may also emit dotted aliases:

- `modal.opened` for `modal-opened`
- `modal.updated` for `modal-updated`
- `modal.closed` for `modal-closed`

A modal detail object should be safe to expose to the embedding page:

```
{
  id: "modal-42",
  type: "notice",
  title: "Safari Permissions",
  message: "...",
  url: null,
  link: {
    title: "View Bug",
    href: "https://bugs.webkit.org/show_bug.cgi?id=189503",
    target: "_blank"
  },
  actions: [
    { id: "close", label: "OK", role: "close" }
  ],
  defaultAction: "close",
  dismissAction: "close",
  requiresInput: false
}
```

The public modal methods are:

```
const modal = await webview.getCurrentModal();

await webview.respondToModal({
  id: modal.id,
  action: modal.dismissAction
});

await webview.dismissModal();
```

The low-level API equivalents use the same method names:

```
await webview.callApi('getCurrentModal');
```

```
await webview.callApi('respondToModal', { id, action, value });
await webview.callApi('dismissModal', { id });
```

`dismissModal()` activates the modal's safest advertised dismissal action. The selection order is:

1. use the action whose `id` is `dismissAction`, when present;
2. otherwise prefer an action with role `cancel`;
3. otherwise prefer role `close`;
4. otherwise prefer role `deny`;
5. otherwise, if there is only one action, use that action;
6. otherwise reject with an ambiguous-action error.

Prompt-like modals can accept a value through `respondToModal`:

```
await webview.respondToModal({
  id: "modal-43",
  action: "ok",
  value: "customer supplied text"
});
```

Unsupported or sensitive operations are not required to be advertised as actions. `dismissModal()` still works whenever the modal has a cancel, close, deny, or single remaining action.

The webview policy capability names are `modals.read` for modal lifecycle events and `modals.respond` for `respondToModal()` and `dismissModal()`. Both are enabled by default for the canonical webview policy.

7 Setup and Backend Modes

7.1 `bbx setup`

Customer-facing setup shape:

```
bbx setup [--port|-p <port>] [--hostname|-h <hostname>]
          [--token|-t <token>] [--zeta|-z]
          [--backend <http|https>]
          [--flipbook-record <dir>]
          [--flipbook-description <text>]
          [--for <user>|--for=<user>]
```

- `-port` chooses the main BrowserBox service port.
- `-hostname` sets the hostname customers use.
- `-token` sets the login token explicitly instead of generating one.
- `-zeta` enables host-per-service mode.
- `-backend http|https` selects whether BrowserBox itself serves HTTP or HTTPS.
- `-flipbook-record` enables flipbook recording (see Section 17).
- `-flipbook-description` sets an optional description for the recording.

- `-for` runs setup on behalf of an unprivileged user (see Section 9). Both `-for <user>` and `-for=<user>` are accepted.

  Runtime values are written to:

```
~/.config/dosaygo/bbpro/test.env
```

7.2 Persistent configuration overrides (`user.env`)

Running `bbx setup` regenerates `test.env`, which overwrites any values you added there directly. To make overrides that survive a re-setup, place them in:

```
~/.config/dosaygo/bbpro/user.env
```

BrowserBox sources `user.env` after `test.env` every time it starts, so values set there take precedence and are never touched by `bbx setup`. The file is not created automatically—create it once and it persists indefinitely.

Example:

```
# ~/.config/dosaygo/bbpro/user.env
# Overrides that survive bbx setup regeneration.
export BBX_MAX_TABS=4
export BBX_SHUTDOWN_IDLE_MS=3600000
```

Any valid environment variable accepted by BrowserBox may be placed in `user.env`. The file is loaded by both `bbx start` (`bbx.sh`) and the service runner (`_conrun.sh`).

7.2.1 Recommended high-throughput profile

The following profile is a good starting point for high-throughput deployments on well-provisioned hosts. It keeps adaptive imagery enabled so BrowserBox continuously tunes screencast quality and frame cadence to the connection, and keeps WebRTC enabled so BrowserBox can use it when it is the best available transport and fall back to WebSocket when conditions change. On more constrained or high-latency hosts, keep `BBX_LOW_END_MODE` at its default and tune settings to your workload.

```
# Keep adaptive quality and frame-rate control active.
export BBX_ADAPTIVE_IMAGERY=true

# Avoid Chrome's forced low-end behavior on adequately provisioned hosts.
export BBX_LOW_END_MODE=false

# Optional when the deployment does not need BrowserBox audio.
export BBX_NO_AUDIO=true

# GPU hosts only; requires working vendor EGL/DRI userspace and permissions.
export BBX_GPU=true
```

Do not set `BBX_DISABLE_WEBRTC=true` in this profile. `BBX_GPU` is an explicit bare-metal hardware opt-in, not an automatic default, and it carries three prerequisites — all of them must hold before the flag is retained in production:

1. An actual hardware GPU is attached to the machine (for cloud instances, a GPU-equipped shape such as one with an NVIDIA L4 or T4).

2. The correct vendor drivers and EGL/DRI userspace are installed and the BrowserBox user has permission to access the device.
3. Chrome on that machine verifies acceleration is live: browse to `chrome://gpu` inside the BrowserBox session and confirm the Graphics Feature Status entries report green *"Hardware accelerated"*. If entries report "Software only" or SwiftShader appears as the renderer, the flag is not delivering hardware acceleration and should be removed.

Docker always disables GPU acceleration and ignores `BBX_GPU`. Diagnostic settings such as `BBX_DIAG` and `BBX_CDP_METRICS` are intentionally omitted here because they are troubleshooting tools rather than production performance settings.

7.2.2 Cross-origin iframe instrumentation

Some pages embed third-party content in cross-origin iframes. Chrome may place those frames in separate renderer processes. BrowserBox now attaches to those targets by default so its supported in-frame capabilities remain available:

```
# Temporary compatibility fallback for a problematic site:
export BBX_OOPIF=false
bbx restart
```

Keep the default enabled unless diagnosing a specific compatibility issue. Disabling it removes BrowserBox instrumentation from separately rendered cross-site frames. See Section 4.4 for the persistent setting and support guidance.

7.3 Reverse proxy and HTTP backend

If BrowserBox sits behind nginx, Caddy, or another edge proxy that terminates TLS:

```
bbx setup --backend http
```

The older compatibility form is still available:

```
bbx setup --http-only
```

7.4 Reverse proxy: preserving service origins (audio and DevTools)

BrowserBox is a multi-service architecture. In addition to the main browser service, it runs separate services for audio streaming and remote DevTools. These services communicate over distinct ports and require their own origin (hostname + port combination) to function correctly.

When all services are proxied through a single origin, audio and DevTools will stop working. This is a browser security constraint: the audio service uses a protocol that requires a unique origin, and the DevTools bridge is likewise origin-sensitive.

7.4.1 The DNS facade approach (recommended)

The recommended pattern is to run each service on its own subdomain, all served over port 443, using wildcard DNS:

```
# Wildcard DNS: *.example.com -> same server
# nginx: route by hostname to the right backend port
#   p8888.example.com -> :8888  (main)
```

```
#   p8886.example.com -> :8886  (audio)
#   p9222.example.com -> :9222  (devtools)
```

BrowserBox automatically detects this pattern (hostname first label matching `p<port>`) and rewires audio and DevTools URLs to the correct subdomain at port 443. Enable it with zeta mode:

```
bbx setup --hostname example.com --port 8888 -z
bbx ng-run
```

7.4.2 Direct multi-port exposure (alternative)

If DNS subdomain routing is not available, expose each port directly from the edge without rewriting the request to a shared origin. Each port must be reachable as its own origin from the client browser.

7.4.3 Minimal mode (no audio or DevTools)

For environments where audio and DevTools are not needed, run in minimal mode to avoid starting those services entirely:

```
export BBX_MINIMAL_MODE=true
bbx start
```

7.5 External TLS hint

If BrowserBox is behind an edge that terminates HTTPS and BrowserBox itself is serving plain HTTP, the runtime can be told to treat the frontend as secure:

```
export BBX_EXTERNAL_TLS=true
bbx start
```

This is useful when a reverse proxy or cloud edge is doing TLS termination for customer traffic.

8 Tunnel and Network-Oriented Commands

8.1 `bbx ng-run`

```
bbx ng-run
```

This is the nginx-oriented workflow. It will trigger zeta-mode setup when needed, expects `hosts.env` support for per-service hostnames, runs nginx setup, and then starts BrowserBox.

Route domain selection. `bbx ng-run` creates per-service hostnames under a route domain. Because of that, `localhost` is not a valid `ng-run` route domain: names such as `audio.localhost` or `devtools.localhost` are not portable DNS targets and should not be relied on. `localhost` remains valid for the direct `bbx run` and `bbx start` workflows.

For local nginx testing, use a wildcard-safe local name such as `bbx.test` or `ci.test`:

```
bbx setup --hostname bbx.test --port 11111 -z
bbx ng-run
```

`ng-run` chooses its route domain in this order:

1. An explicit caller override, such as `DOMAIN=example.test bbx ng-run`.
2. The saved `DOMAIN` from the previous `bbx setup`.
3. The saved setup hostname, when it is suitable for generated subdomains.

Use `DOMAIN` only when you need to override the saved setup domain for this run:

```
DOMAIN=example.test bbx ng-run
```

If the final resolved route domain is `localhost`, BrowserBox refuses to continue and asks for a wildcard-safe domain instead. This protects nginx routing and certificate generation from creating unusable subdomain URLs.

8.2 bbx tor-run

```
bbx tor-run [--anonymize|--clearnet-only] [--onion|--no-onion]
```

This mode sets up Tor integration and can expose BrowserBox through onion routing.

8.3 bbx zt-run

```
bbx zt-run
```

This is the ZeroTier overlay-network entry point for private-network access.

8.4 bbx cf-run

```
bbx cf-run
```

This is the Cloudflare Tunnel entry point for public HTTPS exposure through Cloudflare's edge.

9 Cross-User Execution (-for)

The `-for` flag lets a privileged operator run BrowserBox commands on behalf of an unprivileged runtime user. This is the recommended deployment pattern for multi-tenant or security-hardened environments where the runtime user should have no `sudo` capability.

9.1 Requirements

- The *operator* must have passwordless `sudo` (`sudo -n`).
- The *target user* must already exist and must **not** have `sudo`.
- Linux only — macOS does not support `-for`.

9.2 Supported commands

```
bbx setup   --for <user> [setup-options]
bbx run     --for <user>
bbx ng-run  --for <user>
bbx stop    --for <user>
bbx status  --for <user>
```

Both `-for <user>` and `-for=<user>` syntax are accepted.

9.3 What happens

1. **Operator-level tasks** (TLS certificate generation, nginx configuration, host file entries, port binding) run as the operator via `sudo`.
2. **Runtime tasks** (BrowserBox services, license certification, browser process) run as the target user via `sudo -u <user>`.
3. SSL certificates generated during setup are automatically `chown`ed to the target user so that the Node.js runtime can read them.
4. Configuration is stored under the target user's home directory (`~/<user>/.config/dosaygo/bbpro/`).

9.4 Example workflow

```
# As the operator (e.g. bbxadmin, with sudo):
bbx setup --port 9090 --hostname app.example.com -z --for bbxruntime
bbx ng-run --for bbxruntime

# Later:
bbx status --for bbxruntime
bbx stop   --for bbxruntime
```

All BrowserBox processes will run as `bbxruntime`. The operator never needs to `su` or log in as the runtime user.

9.5 Combining with certificate override

`-for` respects `BBX_SSLCERTS_DIR` when set explicitly:

```
export BBX_SSLCERTS_DIR="/opt/shared-certs"
bbx ng-run --for bbxruntime
```

If `BBX_SSLCERTS_DIR` is not set, certificates are generated into the target user's default location (`~/<user>/sslcerts/`).

10 Zeta Mode and `hosts.env`

The `-zeta` option enables host-per-service mode. BrowserBox then expects:

```
~/.config/dosaygo/bbpro/hosts.env
```

At runtime, `bbx run` reads `ADDR_<PORT>` from that file to construct the login link for the main service port.

Use zeta mode when:

- nginx is doing hostname-based proxying
- you want separate hostnames per BrowserBox service
- you want clean standard-port edge routing

If `hosts.env` is missing, `bbx run` fails fast in zeta mode.

11 Fleet: A Pool of Ephemeral Sessions (`bbx fleet`)

Fleet turns a sufficiently large Linux machine into a pool of ephemeral, clean-slate BrowserBox sessions. It is the machine-level deployment mode for applications that need to hand each authenticated end user a fresh, disposable browser session — without creating static BrowserBox tokens per user, and without your application ever managing Linux users, ports, nginx, or browser profiles.

Fleet is **Linux only** and requires a privileged operator (root, or a non-root account with passwordless `sudo`). Fleet seat users themselves never receive `sudo`.

11.1 The mental model

Do *not* mint static BrowserBox tokens for every end user of your application. Instead, your application allocates a session at the moment it is needed:

```
application authentication succeeds
        |
bbx fleet acquire --json
        |
redirect user to login_url
        |
session closes or expires
        |
bbx fleet release <allocation_id>
```

The login token inside `login_url` is a fresh, single-session capability — it is not your application's user identity. Your application remains responsible for its own authentication (Turnstile/CAPTCHA, SAML or OIDC, JWT validation, authorization), for its own session-timeout and disconnect detection, and for calling `fleet release` when a session ends. Fleet's runtime monitor is a recovery path for BrowserBox processes that exit independently; it does not replace application-level release. Fleet handles everything below that line: Linux seat users, BrowserBox setup, ephemeral tokens, ports, nginx routing, startup, shutdown, clean-slate profile lifecycle, and local allocation locking.

11.2 The three layers

- **Fleet deployment** — machine-level configuration: the seat pool, port range, public hostname and routing topology, wildcard TLS, fleet-wide BrowserBox defaults, clean-slate policy.
- **Fleet seat** — a reusable OS identity (e.g. `bbx-seat-0000`) that can run exactly one allocation at a time. Seats are an implementation detail; integrate against allocation IDs, not usernames.
- **Fleet allocation** — one ephemeral session: a reserved seat, a reserved port set, a fresh login token, a clean browser profile, one public login URL, and one opaque allocation ID.

11.3 Requirements

- Linux, with root or passwordless `sudo` for the operator.
- An installed BrowserBox and a license with enough seats for your target concurrency.
- Machine sizing appropriate to concurrency (each live session runs a real Chrome).
- For production subdomain routing: wildcard DNS (`*.your-domain`), a wildcard TLS certificate, nginx, and public port 443.

- An internal port range for BrowserBox backends (default 7000–20000, loopback only in subdomain mode).

11.4 Initialization

```
sudo bbx fleet init \
  --size 100 \
  --domain browser.example.com \
  --routing subdomain \
  --subdomain-mode random \
  --cert-file /etc/browserbox/fullchain.pem \
  --key-file /etc/browserbox/privkey.pem
```

Initialization is idempotent: re-running it creates only missing seat users, increasing `-size` adds seats and expands routing atomically, and decreasing `-size` never deletes users (excess seats simply become ineligible for new allocations).

Each seat receives a *stable* main port and, in subdomain mode, *stable* public hostnames, so nginx is configured completely at init time — `acquire` and `release` never touch DNS, certificates, or nginx.

11.5 DNS

Subdomain routing requires a wildcard record pointing at the machine:

```
Type: A
Name: *.browser.example.com
Value: 203.0.113.10
```

Add a matching AAAA record if the machine serves IPv6. `fleet init` prints the exact records required, then validates two distinct probe subdomains before touching nginx. If your traffic deliberately arrives through a load balancer or proxy (so DNS does not resolve directly to this machine), pass `-allow-proxied-domain`; Fleet then distinguishes "DNS resolves successfully" from "DNS is confirmed to resolve directly to this machine" rather than silently waiving validation.

11.6 Certificates

Subdomain routing needs a certificate covering `*.your-domain` (and ideally the base domain). Supply an existing wildcard certificate with `-cert-file`/`-key-file`; Fleet validates existence, key permissions, cert/key match, expiry, and wildcard SAN coverage. Because wildcard issuance requires DNS validation, Fleet does not attempt automated wildcard issuance — issue the certificate through your DNS provider (for example `certbot` DNS-01) and hand the files to `fleet init`. Local/test domains (`*.test`, `*.localhost`, etc.) get locally generated certificates automatically.

11.7 Routing modes

- **`subdomain`** (production default): every session is presented through port 443 under a per-seat hostname. Label styles: `port` (`p7000.domain`), `seat` (`seat-0000.domain`), or `random` (`k7p4wm9f.domain` — recommended: stable opaque hostnames, clean URLs). The hostname is stable per seat across allocations; the login token is fresh for every acquisition.

- **direct-port**: the returned URL uses the machine hostname and the seat's port directly (`https://host:7000/login?token=...`). Suitable for testing, internal deployments, or environments with another upstream routing layer. May require opening the port range in your firewall.

Routing is configured fleet-wide, not per acquisition: this keeps the session-start path fast (no DNS change, no certificate operation, no nginx reload) and keeps public URLs stable and auditable. Changing routing topology is a deployment-wide operation — `bbx fleet routing apply` refuses to run while allocations are active unless you explicitly pass `-allow-active`.

11.8 Acquire and release

```
session_json="$(sudo -n bbx fleet acquire --json)"

allocation_id="$(jq -r '.allocation_id' <<<"$session_json")"
login_url="$(jq -r '.login_url' <<<"$session_json")"

# Redirect the already-authenticated application user to $login_url.

sudo -n bbx fleet release "$allocation_id"
```

The supported integration contract is exactly `allocation_id` and `login_url`. In `-json` mode, stdout carries a single JSON document (diagnostics go to stderr) and errors carry stable codes such as `fleet_exhausted`, `fleet_license_unavailable`, or `fleet_public_route_failed`. Acquisition succeeds only after the public login URL is verified live; any failure (license, setup, startup, routing) rolls the local reservation back.

11.9 Clean-slate security

Every allocation starts from a fresh browser profile by default (`FLEET_CLEAN_SLATE=true`), using BrowserBox's canonical `BBX_CLEAN_SLATE` mechanism. Cookies, local storage, history, cache, saved authentication, IndexedDB, and service-worker state from a prior user of the same seat are not intended to survive seat reuse: release stops the runtime, removes the login capability, and wipes the browser profile; acquisition additionally starts with clean-slate enforced. Because different external users receive the same reusable seats over time, do not disable clean slate in multi-user deployments. Persistent per-user profiles are deliberately not part of this design.

11.10 Operational commands

```
sudo bbx fleet list              # active allocations (no secrets)
sudo bbx fleet status [<id>]     # fleet summary or one allocation
sudo bbx fleet reap              # one dead-runtime recovery pass
sudo bbx fleet monitor           # continuous foreground recovery
sudo bbx fleet doctor            # environment / DNS / cert health
sudo bbx fleet reconcile --fix   # repair state drift safely
sudo bbx fleet routing show      # seat-to-hostname-to-port map
sudo bbx fleet routing apply     # atomic whole-of-fleet re-apply
sudo bbx fleet config set K V    # fleet-wide BrowserBox defaults
```

`fleet status` reports local capacity, fully down and partially healthy running allocations, and an *advisory* license-vacancy snapshot; local free seats do not guarantee globally free license seats — license certification during acquire remains authoritative.

11.11 Automatic dead-runtime recovery

BrowserBox can shut itself down after its last client disconnects; the main timeout is controlled by `BBX_SHUTDOWN_IDLE_MS`. Because this shutdown happens inside the seat runtime, Fleet confirms the runtime is fully down and then returns the allocation through the same stop, login-capability removal, and clean-slate profile-wipe path used by an explicit `fleet release`.

Run the foreground monitor under your normal service supervisor:

```
# Persist the BrowserBox idle timeout for every Fleet seat (15 minutes).
sudo bbx fleet config set BBX_SHUTDOWN_IDLE_MS 900000

# Run this foreground process under systemd, Docker, or your supervisor.
sudo bbx fleet monitor
# Optional:
sudo bbx fleet monitor --interval 5 --grace 15
```

A running allocation is eligible for automatic release only when its main port is no longer listening and the seat has no live BrowserBox or Chrome process. Zombie process-table entries do not count as a live runtime. A partial result (only the port or only a live process remains), or an unknown result caused by a failed process-table probe, is reported by `fleet list` and `fleet status`, but is not automatically released. The monitor observes process and socket health outside the short Fleet ownership lock, waits through the grace window, and then revalidates the allocation identity, state, runtime marker, and health before cleanup. A recovered or replaced runtime is therefore left alone, and a slow health probe does not block a simultaneous acquire. Stopping the monitor during that window cancels the in-progress pass rather than shortening the grace.

`fleet reap` performs one such pass. On its normal path, `fleet acquire` makes one locked pass over the seat and allocation records, reserves one seat atomically, and performs startup outside the lock; it does not scan the process table or wait for monitor work. If the records say the pool is exhausted, acquire performs one bounded recovery pass using the shorter acquire grace, retries reservation once, and then reports exhaustion. This just-in-time fallback recovers capacity if the monitor is temporarily unavailable while keeping the ordinary acquire path lightweight. Two simultaneous acquires cannot reserve the same seat or overlapping port set, and simultaneous releases are idempotent.

The monitor is quiet when no recovery candidates exist. A pass with candidates emits one summary rather than one success line per allocation. Failure detail is capped (10 allocations per pass by default), additional detail is summarized as suppressed, and repeated failed passes use exponential backoff capped at 300 seconds. Stale `reserved`, `starting`, or `releasing` records are owned by monitor/reap recovery after their transition TTL (900 seconds by default); acquire does not perform heavy stale-state repair on its normal path.

11.12 Security guidance

- `login_url` is a secret capability. Do not log it; deliver it to the authenticated user and forget it. Retain only the `allocation_id`.
- Call `fleet release` on session timeout *and* explicit logout.
- Fleet seat users must never have `sudo`.
- Fleet state (`~/.config/dosaygo/bbpro/fleet/`) is operator-private (0700/0600).
- Human-readable `fleet list` output never shows tokens; `-json` output (which does include `login_url`) is for the privileged operator only.

11.13 Reference architecture

A typical hardened deployment in front of Fleet:

```
Cloudflare Tunnel / reverse proxy
        |
Turnstile
        |
SAML identity provider
        |
application authorization and JWT
        |
bbx fleet acquire          (this machine)
        |
BrowserBox public Fleet hostname (port 443)
        |
target pension/member portal
```

The SAML/JWT system and Fleet are separate layers: your application proves who the user is; Fleet hands that user one disposable browser session and takes it back afterwards.

12 TLS and Certificates

12.1 BrowserBox-managed TLS

On normal local or self-hosted setups, `bbx setup` ensures host resolution is sane, obtains certificates through BrowserBox's TLS flow, and starts BrowserBox in HTTPS mode when the expected certificate files exist.

BrowserBox automatically chooses the certificate strategy based on the hostname:

- **Domain names with public DNS** — Let's Encrypt via `certbot` (`http-01` challenge). An email address is required.
- **Public IP addresses** (e.g. `5.161.100.137`) — Let's Encrypt short-lived certificates (6-day validity) via `acme.sh`. Port 80 must be reachable. Falls back to `mkcert` if LE issuance fails.
- **Private/loopback IPs** (`127.x`, `10.x`, `192.168.x`, `172.16-31.x`) — locally trusted certificates via `mkcert`.
- **Local hostnames** (`localhost`, `*.local`, `*.test`, etc.) — locally trusted certificates via `mkcert`.

Example with a public IP:

```
bbx setup --hostname 5.161.100.137 --port 8888
# Obtains a Let's Encrypt short-lived cert (auto-renewed by acme.sh)
bbx start
```

12.2 Customer-managed certificates

If you want BrowserBox to use your own certificate material instead of BrowserBox-managed certificates, set:

```
export BBX_SSLCERTS_DIR="/path/to/your/certs"
```

The certificate directory must contain:

- `fullchain.pem`
- `privkey.pem`

When `BBX_SSLCERTS_DIR` is set and the directory already contains `fullchain.pem` and `privkey.pem`, BrowserBox skips all certificate generation (no Let's Encrypt, no mkcert) and uses the provided certificates as-is.

When `BBX_SSLCERTS_DIR` is set but the directory does not yet contain certificates:

- `bbx setup` and `bbx ng-run` generate certificates into that directory instead of `~/sslcerts/`.
- Nginx configuration points to the override directory.
- The Node.js runtime reads certificates from there.
- In `-for` mode (Section 9), certificates are generated into the override directory and `chown`ed to the target user.

If you already have externally managed certificates, simply point `BBX_SSLCERTS_DIR` at the directory—BrowserBox detects existing certs and skips generation automatically.

For `bbx ng-run`, BrowserBox generates per-service names as `<random>.<domain>`. The certificate must cover those generated names. For example:

```
BBX_HOSTNAME=fpf-dev-bbox.fdnypen.local
required nginx SAN: *.fpf-dev-bbox.fdnypen.local
```

A shallower wildcard such as `*.fdnypen.local` only covers one label under `fdnypen.local`. When `BBX_SSLCERTS_DIR` points at an existing certificate and exactly one shallower wildcard parent matches the configured hostname, BrowserBox uses that parent for `ng-run` route generation and logs the decision. You can also make the parent explicit:

```
export DOMAIN="fdnypen.local"
bbx ng-run --for bbxruntime
```

Another common TLS pattern is to run `bbx setup -backend http` and terminate TLS in nginx, Caddy, or another reverse proxy.

13 Customer-Relevant Environment Variables

13.1 Core customer-facing variables

Name	Default	Meaning
ALLOWED_EMBEDDING_ORIGINS	unset	Space-separated allowlist of parent origins permitted to embed BrowserBox.
BBX_HTTP_ONLY	unset	Forces HTTP-only mode and skips HTTPS certificate generation. Useful behind a reverse proxy.
BBX_EXTERNAL_TLS	unset	Treats the frontend as secure when TLS is terminated upstream.

BBX_SSLCERTS_DIR	unset	Directory containing customer-managed TLS files such as `fullchain.pem` and `privkey.pem`. Respected by `setup`, `ng-run`, and `-for` mode.
BBX_SKIP_CERT_COPY	unset	Prevents BrowserBox-managed certificates from overwriting certificates in the configured certificate directory.
BBX_SHUTDOWN_IDLE_MS	2700000	Main idle shutdown after all clients disconnect.
BBX_MINIMAL_MODE_IDLE_MS	120000	Minimal-mode idle shutdown fallback.
FLEET_REAP_INTERVAL_SECS	5	Delay between foreground `fleet monitor` recovery passes.
FLEET_REAP_GRACE_SECS	15	Required continuously-down window before monitor or one-shot reaping releases an allocation.
FLEET_ACQUIRE_REAP_GRACE_SECS	2	Confirmation window for acquire-time recovery of allocations already observed fully down.
FLEET_STALE_RESERVE_SECS	900	Age after which a fully down transitional allocation is eligible for monitor/reap recovery.
FLEET_MONITOR_MAX_BACKOFF_SECS	300	Maximum retry delay after repeated monitor-pass failures.
FLEET_MONITOR_FAILURE_DETAIL_LIMIT	10	Maximum per-allocation automatic-release failure lines in one pass; additional failures are summarized.
BBX_HOME_PAGE	unset	Preferred home page URL.
BBX_DEFAULT_HOME_PAGE	https://duckduckgo.com	Fallback home page URL.
BBX_CUSTOM_CURSOR	unset	Path to a local image file used as the remote cursor image.
BBX_HOSTNAME	system hostname or prompt	Hostname used for BrowserBox setup.
DOMAIN	unset	Explicit route-domain override for network-oriented commands such as `ng-run`. Use only when overriding the saved setup domain.
BBX_UI_THEME_OVERRIDE	unset	Default login UI theme when a supported theme is requested.

`BBX_SHOW_STREAMING_STATS_OVERLAY`	unset	Displays the live streaming stats overlay in the BrowserBox client UI.
BBX_MAX_CONNECTIONS	5	Per-session client connection cap.
BBX_MAX_TABS	20	Maximum concurrent page tabs per BrowserBox session. Applies to both user-initiated tab creation and browser-initiated popups (e.g. `window.open` or links with `target="_blank"`). When a client is connected at the time a popup is blocked, a notification appears in the client UI. Tabs that would exceed the cap at service startup are closed silently, with no client notification. Set this persistently in `user.env` (see Section 7.2) so it survives `bbx setup` regeneration.
BBX_CLEAN_SLATE	unset	When `true`, clears the browser user-data directory at startup. Useful for ephemeral deployments that need a fresh profile every launch.
BBX_MINIMAL_MODE	unset	Runs BrowserBox with only the main service (no audio, docs, or devtools sidecars). Useful for constrained environments or single-purpose kiosks.
BBX_DISABLE_WEBRTC	unset	When `true`, disables WebRTC entirely and forces WebSocket-only transport.
BBX_AD_BLOCK	true	Master switch for ad-specific blocking. Set to `false` and restart BrowserBox to disable both BrowserBox request-level ad blocking and Chrome's native ad filter. Policy enforcement, authentication, and PDF handling may still use request interception without blocking ads.
`BBX_NATIVE_AD_BLOCK`	false	Adds Chrome's native subresource ad filter when `true`. Has no effect when `BBX_AD_BLOCK=false`.
BBX_ADAPTIVE_IMAGERY	true	Adapts screencast quality and frame cadence from measured streaming pressure. Leave enabled for the measured high-throughput profile.

BBX_LOW_END_MODE	true	When `true`, forces Chrome low-end-device behavior. Set to `false` on adequately provisioned performance hosts; validate constrained deployments before changing it.
BBX_NO_AUDIO	unset	When `true`, does not start the audio service and tells the client not to retry audio setup. Use only when remote audio is not required.
BBX_NETWORK_DOMAIN	false	Controls fine-grained CDP Network tracking. Disabled by default: BrowserBox synthesizes page-loading progress from page lifecycle events, avoiding a high-volume internal event stream on network-heavy applications. Set to `true` to restore fine-grained network telemetry. Navigation to local files and browser-internal pages remains blocked in both modes.
BBX_OOPIF	true	Attaches BrowserBox to Chrome's separately rendered cross-site iframe targets so cursor and supported page integrations work inside OOPIFs. Set to `false` and restart only as a temporary compatibility fallback.
`BBX_CDP_PERMESSAGE_DEFLATE`	false	Controls compression on the trusted loopback Chrome DevTools WebSocket. Keep disabled to avoid compression CPU overhead.
`BBX_CDP_SKIP_UTF8_VALIDATION`	true	Skips redundant UTF-8 validation on trusted loopback CDP JSON messages. Set to `false` for compatibility diagnostics.
`BBX_CDP_ALLOW_SYNCHRONOUS_EVENTS`	true	Advanced tuning. Controls whether multiple CDP WebSocket messages may be delivered in a single event-loop turn. Setting `false` gives the Node event loop more room to breathe under heavy CDP load, at the cost of slightly higher per-message latency.

BBX_GPU	unset	On supported bare-metal Linux hosts, `true` enables the hardware-GPU profile, including explicit headless GPU enablement, native EGL ANGLE, and GPU rasterization. Docker always disables GPU acceleration and ignores this variable.
LICENSE_KEY	unset	Your BrowserBox product key. Can be set in the environment, passed to `bbx certify`, or persisted in `test.env` after first certification.

13.2 Examples

```
export ALLOWED_EMBEDDING_ORIGINS="https://app.example.com https://localhost:*"
export BBX_HTTP_ONLY=1
export BBX_EXTERNAL_TLS=true
export BBX_SSLCERTS_DIR="/path/to/certs"
export BBX_SKIP_CERT_COPY=1
export BBX_HOME_PAGE="https://intranet.example.com"
export BBX_CUSTOM_CURSOR="/absolute/path/to/cursor.png"
export BBX_SHUTDOWN_IDLE_MS=7200000
export BBX_UI_THEME_OVERRIDE=dark
export BBX_SHOW_STREAMING_STATS_OVERLAY=true
export BBX_MAX_CONNECTIONS=8
export BBX_MAX_TABS=24
bbx setup
bbx start
```

Cross-user execution:

```
# Run as operator, services execute as bbxruntime
bbx setup --port 9090 --hostname app.example.com -z --for bbxruntime
bbx ng-run --for bbxruntime
bbx stop --for bbxruntime
```

Certificate override with cross-user execution:

```
export BBX_SSLCERTS_DIR="/opt/shared-certs"
bbx ng-run --for bbxruntime
```

14 Licensing and Occupancy Visibility

14.1 Available today

- `bbx certify` validates the license and reserves a seat when possible.
- `bbx vacancy` queries the license server and reports total seats plus current occupied and vacant counts when the server supports occupancy summaries.
- Local reservation metadata is stored under `~/.config/dosaygo/bbpro/tickets/`.

14.2 What `bbx vacancy` means

```
bbx vacancy
```

For modern servers, `bbx vacancy` reports a customer-safe summary:

- total seats
- occupied seats now
- vacant seats now
- leased active
- reserved active

By default it does not expose seat IDs or deeper lease inventory to ordinary customer keys.

14.3 What is not surfaced here

This BrowserBox runtime still does not surface richer customer history such as:

- detailed current lease inventory for ordinary customer keys
- free-seat counts over time
- occupancy history

15 Practical Recipes

15.1 Embedded BrowserBox without the standard UI

```
export ALLOWED_EMBEDDING_ORIGINS="https://app.example.com"
export BBX_HOME_PAGE="https://app.example.com"
bbx setup -p 8888
bbx start
```

Then use a login link shaped like:

```
https://localhost:8888/login?token=...&ui=false
```

15.2 BrowserBox behind a reverse proxy

```
bbx stop
bbx setup --backend http -p 8080 --hostname browserbox.internal
bbx start
```

Then let nginx, Caddy, or a cloud load balancer terminate TLS and forward plain HTTP to BrowserBox.

15.3 Check whether a seat is currently available

```
bbx vacancy
```

This command reports the current seat summary, including total seats, occupied seats, vacant seats, active leases, and active reservations.

Some deeper occupancy reporting may vary by deployment version.

16 Transport and Streaming

BrowserBox uses an adaptive, ACK-paced streaming pipeline to deliver remote browser frames to connected clients. Two transport modes are available:

16.1 WebRTC (default)

By default, BrowserBox negotiates a peer-to-peer data channel between server and client. This delivers the lowest latency and highest frame throughput. The streaming pipeline observes round-trip time and automatically adjusts frame pacing.

16.2 WebSocket fallback

If WebRTC negotiation fails (firewalled UDP, restricted NAT, or `BBX_DISABLE_WEBRTC=true`), BrowserBox falls back to WebSocket binary transport. This works through any HTTP proxy or load balancer and still delivers real-time frames, though with slightly higher latency than WebRTC.

For deployments behind an HTTP proxy or strict corporate firewall where WebRTC is unreliable, forcing WebSocket-only mode improves connection stability:

```
export BBX_DISABLE_WEBRTC=true
bbx start
```

16.3 Adaptive quality and backpressure

The BrowserBox streaming pipeline adapts JPEG quality and frame pacing automatically based on network conditions. Key behaviors:

- JPEG quality is adjusted between a minimum and maximum bound in response to downstream capacity.
- When the client is consuming frames faster than they are produced, quality is increased.
- When frames are backing up (the client is slower than the server), quality is reduced and some frames are dropped to maintain real-time responsiveness.
- The pipeline uses round-trip acknowledgements to detect network pressure and adjust pacing.

16.4 Streaming diagnostics

BrowserBox can show a live streaming stats overlay in the client UI with FPS, transport mode, round-trip time, and frame-flow status.

To display it:

```
export BBX_SHOW_STREAMING_STATS_OVERLAY=true
bbx start
```

The overlay shows live metrics including:

Metric	Meaning

FPS	Frames per second currently being delivered to the client. This reflects actual browser redraw activity—if the remote page is static, the FPS will naturally be low even on a healthy connection.
Transport	Active transport mode: `WebRTC` or `WebSocket`.
RTT	Round-trip time between server and client, used by the pipeline for pacing decisions.
Backpressure	Whether the pipeline is currently throttling or dropping frames to keep up with network conditions.

Important: a low FPS reading does not necessarily indicate a problem. If the remote page content is not changing, BrowserBox does not generate frames—FPS measures browser redraw activity, not a minimum guaranteed stream rate. A high-resolution page with no animations will show near-zero FPS on a healthy connection.

17 Flipbook Recording

BrowserBox can record a browsing session as a *flipbook*—a self-contained static site of sequential JPEG frames with a JavaScript viewer. Flipbook recordings are produced directly from the internal screencast pipeline with negligible overhead: when recording is off, the only cost is a single boolean check per frame.

17.1 Quick start

```
bbx setup --flipbook-record ~/my-recording \
          --flipbook-description "Onboarding walkthrough"
bbx run
# ... use BrowserBox normally ...
bbx stop
```

When `bbx stop` is called, BrowserBox compiles the captured frames into a complete flipbook static site and (if Cloudflare `wrangler` is available) offers to deploy it to Cloudflare Pages.

17.2 How it works

1. `bbx setup` writes `BBX_FLIPBOOK_DIR` and optionally `BBX_FLIPBOOK_DESCRIPTION` to `~/.config/dosaygo/bbpro/test.env`.
2. At runtime, each screencast frame is written as a JPEG + JSON metadata pair to a temporary directory inside the BrowserBox config directory (`~/.config/dosaygo/bbpro/flipbook_recording/`).
3. On `bbx stop`, the built-in `browserbox flipbook-generate` command compiles the raw frames into a flipbook static site under the directory provided by `-flipbook-record`.

When recording is disabled (no `-flipbook-record` flag), **there is zero overhead** on the frame pipeline—only a single falsy-value check is evaluated per frame.

17.3 Output structure

Each recording produces a timestamped directory inside the flipbook base directory:

```
~/my-recording/
  2026-04-14T02-25-00-000Z--2026-04-14T02-30-00-000Z/
    site/
      index.html          # self-contained viewer
      manifest.json       # flipbook v1 manifest
      pages/
        000000.jpg        # contiguous, zero-padded frames
        000001.jpg
        ...
      assets/
        viewer.css
        viewer.js
        sw.js             # service worker for offline caching
      meta/
        provenance.json   # full per-frame metadata + recording info
```

Multiple runs with the same `-flipbook-record` directory produce separate timestamped subdirectories, each a self-contained flipbook site.

17.4 Frame normalization

Frame IDs from the internal pipeline may contain gaps (due to cast restarts or tab switches). The site generator performs a normalization pass: frames are sorted, paired by basename (JPEG with its JSON metadata), and re-indexed as a contiguous zero-padded sequence (`000000`, `000001`, ...). The original frame IDs are preserved in `meta/provenance.json` for traceability.

17.5 Cloudflare Pages deployment

If `wrangler` (the Cloudflare CLI) is available or can be installed, `bbx stop` will offer to deploy the generated flipbook site to Cloudflare Pages:

```
# Install wrangler if not already present
npm install -g wrangler

# Authenticate (one-time)
wrangler login

# Deployment happens automatically on bbx stop
```

The Pages project name is derived from the flipbook directory basename. If `wrangler` is not available, the site is still generated locally and can be served with any static file server.

17.6 Relevant environment variables

`BBX_FLIPBOOK_DIR`	Absolute path to the flipbook output directory. Set by `-flipbook-record`. When empty, recording is disabled.
`BBX_FLIPBOOK_DESCRIPTION`	Optional human-readable description embedded in the manifest and provenance metadata. Set by `-flipbook-description`.

17.7 Direct site generation

The flipbook site generator is available as a standalone command on the BrowserBox binary:

```
browserbox flipbook-generate <flipbookDir>
```

This reads raw frames from the config directory, compiles the flipbook site into a timestamped subdirectory of `<flipbookDir>`, and cleans up the temporary recording data. This is called automatically by `bbx stop` but can also be invoked manually.

18 Future Directions

Likely next customer-facing improvements:

- a published `hosts.env` schema with nginx examples
- a reverse-proxy cookbook for `-backend http`
- a separate customer-safe reference for supported login-link parameters
- path-prefix wildcard support in navigation allowlists and blocklists (e.g. `example.com/path/*`)
- per-destination feature controls so that different `featureControls` values can be applied to different allowlist entries (current controls are global across all destinations)
- a Back/Forward option in the remote-page context menu (currently available only through the embedder webview API)